CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)
Chandubhai S. Patel Institute of Technology (CSPIT)
Department of Computer Engineering

LABORATORY PRACTICAL SUBMISSION RECORD

ITUE203 — Web Development Frameworks

Practicals 1 to 14

Project: Student Hub

Student Name	**Dhruv Bhoi**
Student ID	**D26CE154**
Department	Computer Engineering (CE)
Academic Year / Term	2026 - 2027 (ODD Semester)
Semester	3rd Semester, B.Tech CE
Institute	CSPIT

Table of Contents / Index of Practicals

Practical No.	Practical Title & Topic
Practical 1	Project Initiation, Sitemap, Wireframe, Git & GitHub Setup
Practical 2	Semantic HTML5 Pages with Accessibility-Ready Structure
Practical 3	Responsive UI Design using CSS Grid, Flexbox & Modern SaaS Theme
Practical 4	JavaScript DOM Manipulation, Event Handling & Interactivity
Practical 5	Registration Form with Frontend Validation & Canvas CAPTCHA
Practical 6	Rendering External JSON Data using Fetch API, Search & Filters
Practical 7	PHP Form Processing with Server-Side Validation & CSV/JSON Storage
Practical 8	MySQL Schema Design, ER Model, PDO Connectivity & Diagnostics
Practical 9	Secure User Registration with Duplicate Check & Password Hashing
Practical 10	Secure Login/Logout with Sessions, RBAC, Timeout & Remember-Me
Practical 11	Student Management CRUD with Search, Filter & Soft Delete
Practical 12	Event Management CRUD with Poster Upload & File Validation
Practical 13	PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload
Practical 14	Admin Dashboard with Role-Based Access Control & Audit Log

Practical 1: Project Initiation, Requirement Analysis, Sitemap, Wireframe, and GitHub Setup

Practical Aim & Tools

Aim: Initiate the semester-long Student Hub portal by identifying the problem scope, user roles, key modules, navigation flow, and minimum 10 pages. Create a sitemap, low-fidelity wireframe, project folder structure, README file, and GitHub repository.

Tools & Technologies: VS Code, Draw.io, Git, GitHub, Markdown

1. Project Overview & Problem Statement

StudentHub is a modern, unified campus management web application designed for Charotar University of Science and Technology (CHARUSAT). Historically, college students have had to navigate multiple fragmented portals for attendance, course details, campus events, and administrative notices. StudentHub consolidates these functions behind a single secure login with role-based access control (Student vs. Administrator), presenting users with a sleek modern SaaS dashboard layout featuring timetable cards, circular SVG donut progress rings, and a live notice board feed.

2. User Roles & Modules

• Guest / Public: Landing page (hero slider), about page, event catalog, student directory with dependent state-city filters, FAQ accordion, contact form, and feedback submission.
• Student: Secure authentication, personal timetable, attendance & assignment progress donuts, 1-click event RSVP & cancellation, and student profile management.
• Administrator: Student management CRUD (search, filter, pagination, soft delete/restore, CSV export), Event management CRUD (with poster image uploads), dynamic Admin Dashboard with stored procedure monthly registration bar chart, and user active/inactive toggles.

3. Sitemap & Navigation Architecture

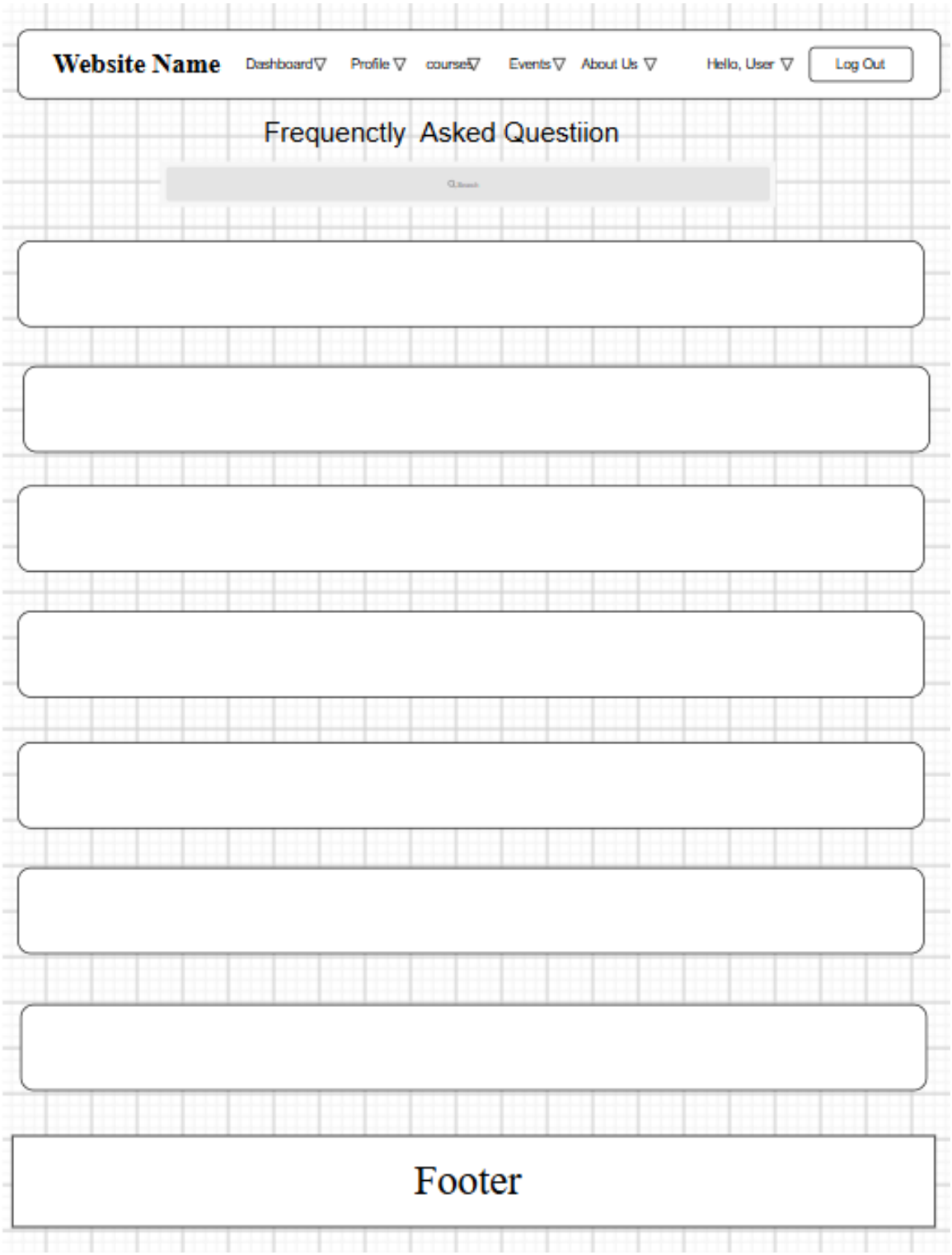


Figure: StudentHub Hierarchical Sitemap & Navigation Map

4. Low-Fidelity Wireframes

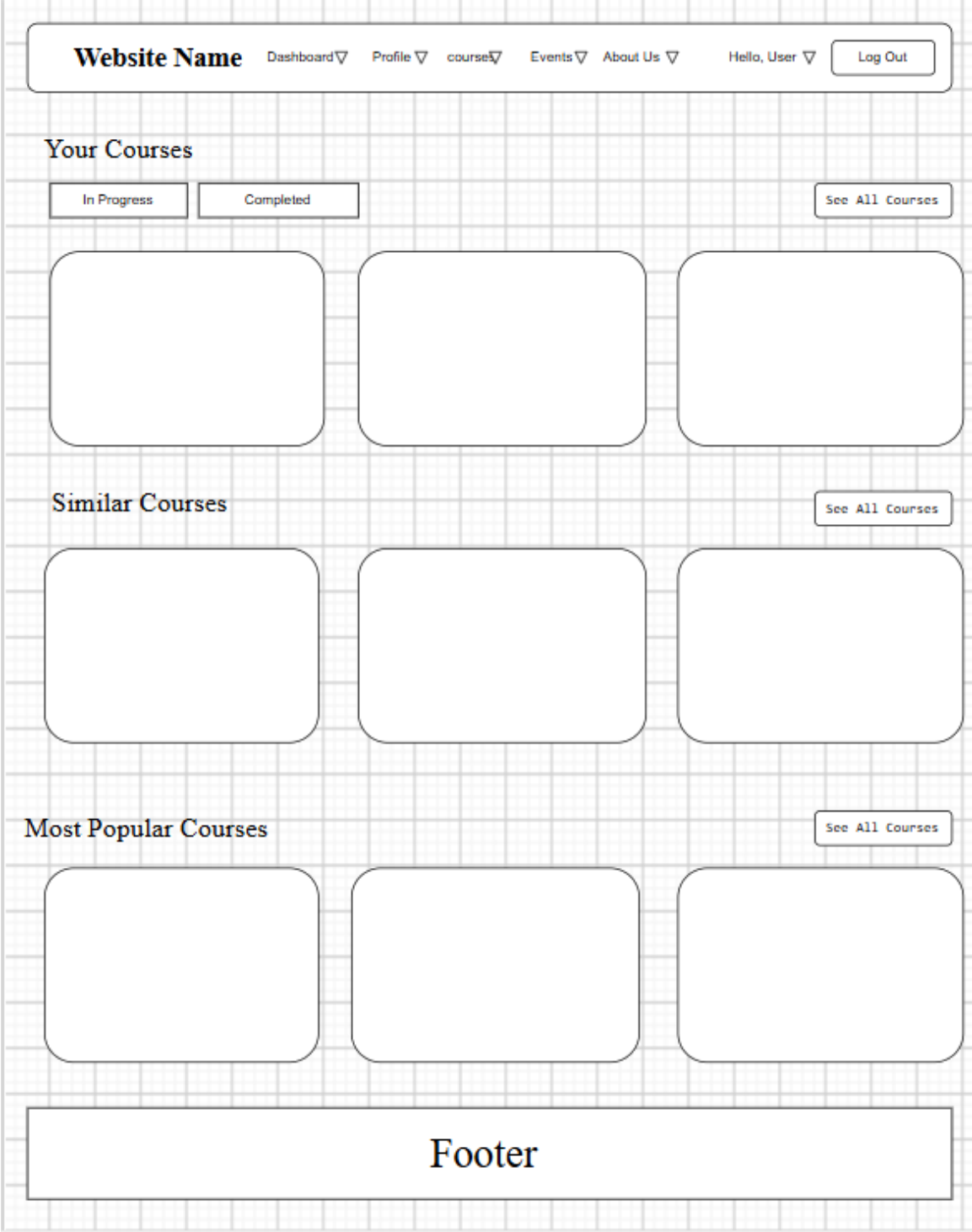


Figure: Low-Fidelity Dashboard Wireframe (Left Sidebar, Schedule Cards & Stat Widgets)

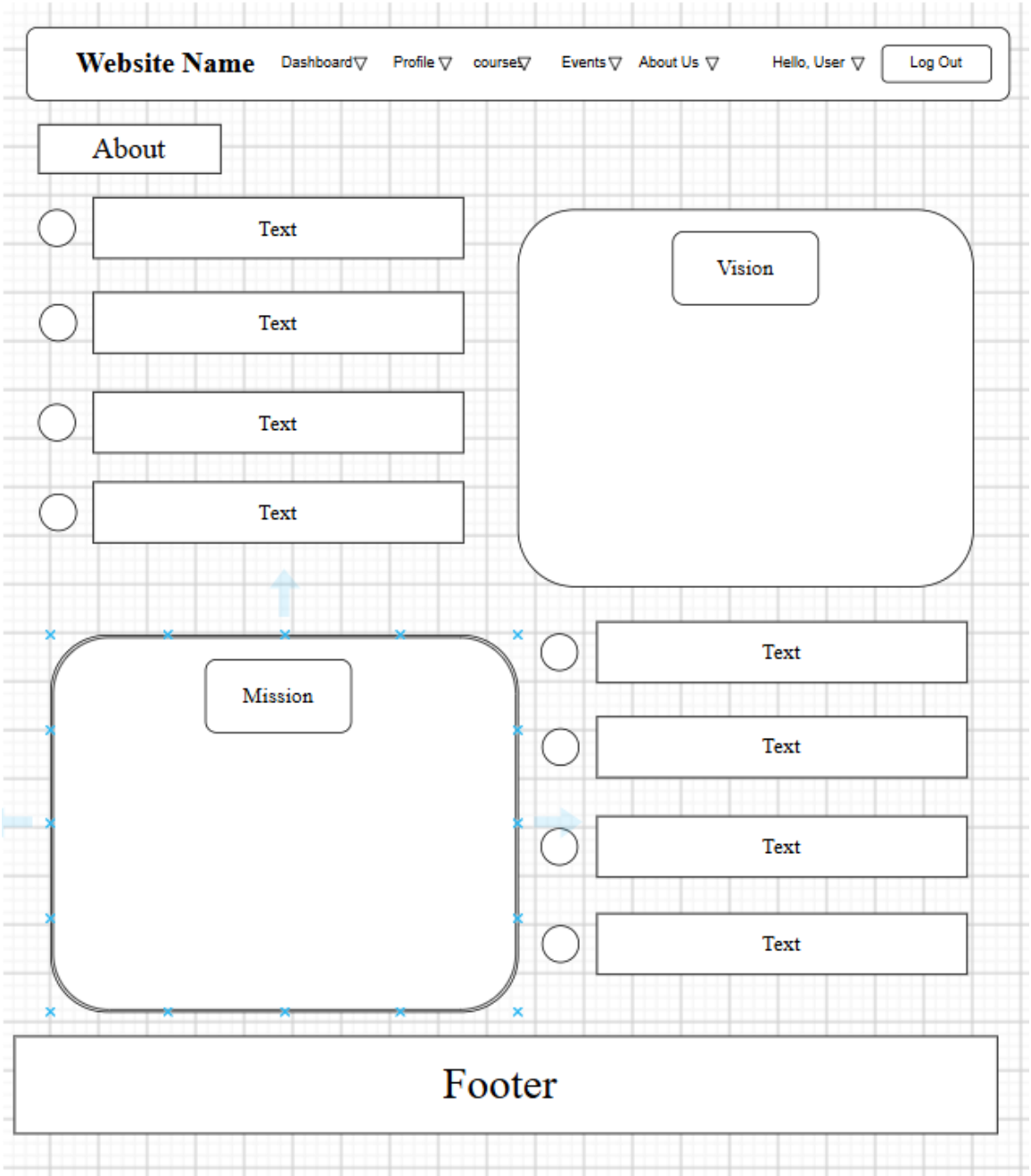


Figure: Low-Fidelity Events Wireframe (Toolbar, Search, Cards Grid & Pagination)

5. Project Folder Structure

Listing: Clean, Production-Ready Directory Structure

```
studenthub/
├── assets/
│   ├── css/style.css           # Acade SaaS design tokens, layout shell, schedule cards
│   ├── js/                     # main.js, validation.js, data-render.js, api-crud.js
│   └── images/                 # logos, banners, placeholders
├── data/
│   ├── events.json             # Sample JSON dataset for campus events
```

```
│   ├── students.json            # Sample JSON dataset for student directory
│   ├── contacts.csv             # Server-side CSV storage for contact inquiries
│   └── feedback.json            # Server-side JSON storage for rating feedback
├── database/
│   └── studenthub.sql           # Normalized 3NF MySQL schema, triggers, seed data
├── docs/                        # requirements.md, sitemap.md, wireframes.md, er-diagram.md
├── includes/
│   ├── config.php               # Environment constants, session lifetime, credentials
│   ├── db.php                   # Secure PDO & MySQLi dual connection helper
│   ├── functions.php            # Auth, CSRF, flash messaging, audit trail
│   ├── header.php               # Acade layout sidebar drawer & top search/greeting bar
│   └── footer.php               # Shared footer attribution & dynamic scripts
├── static/                      # 12 accessible static HTML5 mockups (Practical 2)
├── admin/
│   ├── dashboard.php            # KPI stat cards, stored procedure charts, user toggles
│   └── audit.php                # System activity audit trail log viewer
├── api/
│   └── events.php               # RESTful JSON API endpoint for events CRUD
├── index.php, login.php, register.php, dashboard.php, profile.php, events.php
└── students.php, manage_events.php, api-demo.php, db-test.php
```

6. Key Questions & Analysis

Q: What is a URL and what are its constituent parts?

Ans: A Uniform Resource Locator (URL) is a standardized web address referencing a resource over a network. Its parts include: 1) Scheme/Protocol (http, https), 2) Subdomain & Domain Name (studenthub.local or charusat.edu.in), 3) Port (:80, :443), 4) Path (/events.php), 5) Query String (?category=Workshop&sort=date:asc), and 6) Fragment identifier (#main).

Q: How does a web browser process an HTML file?

Ans: The browser performs: 1) Conversion of raw bytes into characters based on UTF-8, 2) Tokenization into HTML tags, 3) Construction of the Document Object Model (DOM) tree, 4) Processing of linked CSS stylesheets into the CSSOM tree, 5) Combining DOM and CSSOM into a Render Tree, 6) Layout computation (geometric positions of elements), and 7) Painting pixels to screen.

Q: How will page navigation flow be managed among all pages?

Ans: Through a unified, responsive application shell (`includes/header.php`) featuring a fixed left navigation sidebar on desktop that collapses into an accessible slide-out mobile drawer with an overlay. Links dynamically highlight the active page using `aria-current="page"`.

Practical 2: Semantic HTML5 Pages with Accessibility-Ready Structure

Practical Aim & Tools
Aim: Develop static HTML5 skeletons for at least 10 Student_Hub pages such as Home, About, Register, Login, Dashboard, Profile, Events, Directory, FAQ, Contact, Feedback, and Admin using semantic tags with accessibility support.

Tools & Technologies: VS Code, HTML5, WAI-ARIA, Screen Reader Testing

1. Semantic Architecture & WAI-ARIA Landmarks

Modern web standards require semantic HTML5 elements over generic tags to enable screen readers, assistive technology, and search engine spiders to understand page hierarchy. In StudentHub, every page adheres to the following landmark structure:
• <header> & <aside class="app-sidebar">: Application branding, search input, and main navigation landmark.
• <main id="main" class="container">: Primary landmark holding core page content, paired with a <a class="skip-link" href="#main"> for keyboard accessibility.
• <section> & <article>: Grouping logical modules (e.g. event cards, student profile sections, notice boards).
• <footer>: Site metadata, university credentials, and copyright attribution.
• ARIA attributes: aria-live="polite" on dynamic search results and error messages, aria-expanded="false" on accordions and menus, and role="status" on alerts.

2. Minimum 10 Implemented Pages Inventory

StudentHub implements 14 full pages (exceeding the syllabus requirement of 10):
1. index.php (Home / Hero Slider) | 2. about.php (Project & Credentials) | 3. events.php (Catalog & Filters)
4. directory.php (Dependent Filters) | 5. faq.php (Accordion UI) | 6. contact.php (CSV File Storage)
7. feedback.php (Star Rating/JSON) | 8. login.php (Auth & Remember Me) | 9. register.php (Validation & CAPTCHA)
10. dashboard.php (Acade SaaS) | 11. profile.php (User Profile) | 12. students.php (CRUD & CSV Export)
13. manage_events.php (Uploads) | 14. admin/dashboard.php (Charts/RBAC) | 15. db-test.php (Diagnostic Test Suite)

3. Implementation Code: Semantic Shell

Listing: Semantic HTML5 Shell with Skip-Link and WAI-ARIA

```
<!-- Accessible Semantic Structure Skeleton -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>StudentHub | Modern Campus Portal</title>
  <link rel="stylesheet" href="assets/css/style.css">
</head>
<body>
  <a class="skip-link" href="#main">Skip to main content</a>
  <div class="app-layout">
    <aside class="app-sidebar" id="appSidebar" aria-label="Main Navigation">
      <a class="sidebar-brand" href="index.php"><div class="sidebar-brand-
icon">▲</div><span>StudentHub</span></a>
      <nav aria-label="Sidebar Menu">
        <ul class="sidebar-nav">
          <li><a href="dashboard.php" class="nav-item active" aria-
current="page"><i> </i><span>Dashboard</span></a></li>
          <li><a href="events.php" class="nav-item"><i> </i><span>Events</span></a></li>
        </ul>
      </nav>
    </aside>
    <div class="app-main">
      <header class="app-topbar">
        <button class="hamburger" id="hamburger" aria-label="Toggle menu" aria-
expanded="false">☰</button>
        <div class="topbar-search"><input type="search" placeholder="Search events, courses…"
aria-label="Search"></div>
      </header>
      <main id="main" class="container app-content" tabindex="-1">
        <!-- Main page content here -->
      </main>
      <footer class="site-footer"><small>&copy; 2026 StudentHub - Developed by Dhruv Bhoi
(D26CE154)</small></footer>
    </div>
  </div>
</body>
</html>
```

4. Verification Screenshot: Static Home & Navigation

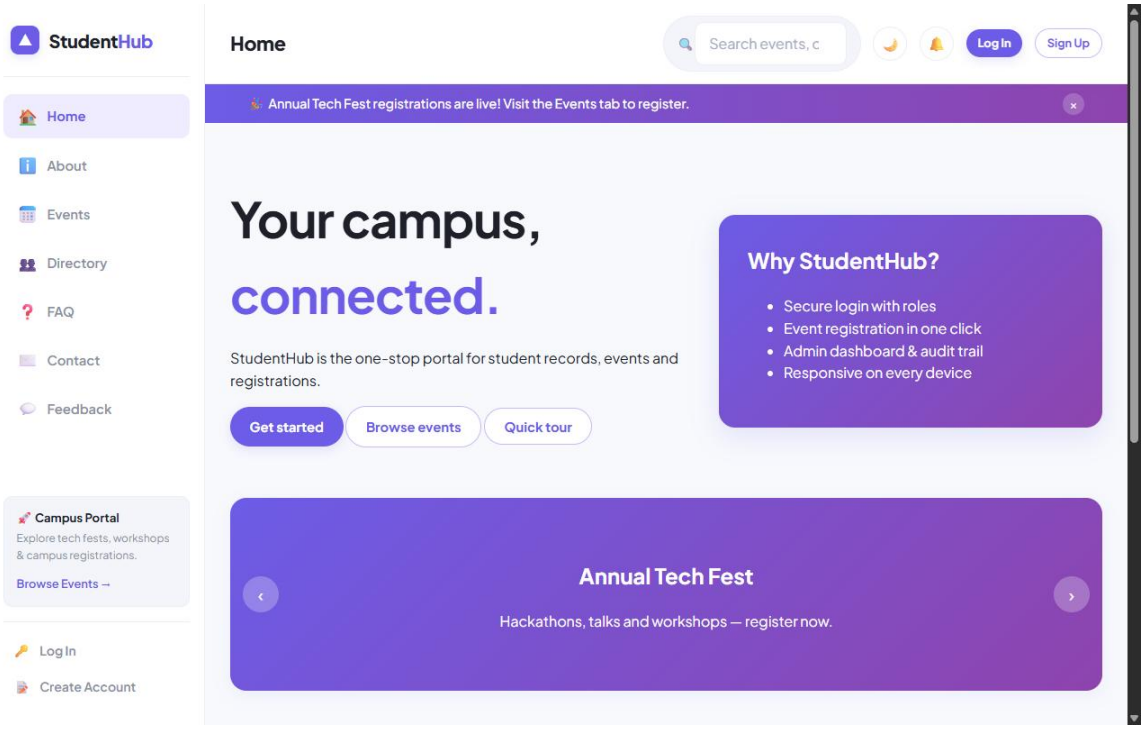


Figure: Semantic Static Landing Page with Accessible Hero and Highlights

5. Key Questions & Analysis

Q: Are semantic elements like header, nav, main, section, article, and footer used properly?

Ans: Yes. StudentHub strictly avoids soup. The page layout uses <header> for titles/tools, <aside> for sidebar navigation, <main> for the single central document body, <article> for self-contained cards (events, reviews), and <footer> for attributions.

Q: Is the website accessibility-ready (WAI-ARIA, skip link, forms)?

Ans: Yes. It implements: 1) Skip link pointing to `#main`, 2) Distinct `<label for="...">` bindings for all form inputs, 3) `aria-live="polite"` regions for live search counts and validation feedback, and 4) `aria-expanded` attributes on interactive menus.

Practical 3: Responsive UI Design using CSS Grid, Flexbox, and Modern SaaS Theme

Practical Aim & Tools
Aim: Design a responsive modern UI using CSS Grid, Flexbox, media queries, CSS variables, and modern SaaS dashboard layout (Acade UI) for mobile, tablet, and desktop screens.

Tools & Technologies: CSS3, CSS Grid, Flexbox, CSS Custom Properties, Responsive Media Queries

1. Design System & CSS Custom Properties

StudentHub utilizes a modern SaaS design system based on a violet/lavender color scheme (`#6C5CE7` primary brand, `#EEECFE` subtle tinted backgrounds, `#2D3436` high-contrast slate text). All design tokens are declared as CSS Custom Properties (`:root`) allowing seamless dynamic dark mode switching.

2. Responsive App Shell & Grid Layouts

• Desktop Layout (≥ 1024px): 2-column flex app shell with a fixed 240px sidebar and fluid `app-main` viewport.
• Timetable Cards: CSS Grid (`grid-template-columns: repeat(auto-fit, minmax(220px, 1fr))`) rendering class schedule cards with lavender badges.
• Circular SVG Progress Rings: Donut cards displaying 92% Attendance and 85% Activity completion with pure CSS stroke-dashoffset animations.
• Mobile Layout (< 768px): The sidebar collapses into an off-canvas drawer controlled via the topbar hamburger button, switching grids into 1-column stacks.

3. Implementation Code: CSS Tokens & Responsive Shell

Listing: Core Responsive CSS Grid & SaaS Design Tokens

```
:root {
  --primary: #6C5CE7;
  --primary-hover: #5A49E0;
  --primary-light: #EEECFE;
  --bg: #F7F7FD;
  --surface: #FFFFFF;
  --text: #2D3436;
  --muted: #8395A7;
  --border: #E8E8F3;
  --radius-lg: 18px;
  --shadow-sm: 0 4px 16px rgba(108, 92, 231, 0.06);
}

.app-layout { display: flex; min-height: 100vh; background: var(--bg); }
.app-sidebar { width: 240px; background: var(--surface); border-right: 1px solid var(--border); }
.app-main { flex: 1; display: flex; flex-direction: column; min-width: 0; }
.app-topbar { height: 70px; background: var(--surface); display: flex; align-items: center;
justify-content: space-between; padding: 0 2rem; }
```

```
/* Responsive Media Queries */
@media (max-width: 1024px) {
  .app-sidebar { position: fixed; left: -260px; z-index: 1000; transition: left 0.3s ease; }
  .app-sidebar.open { left: 0; }
}
@media (max-width: 768px) {
  .grid.cols-3 { grid-template-columns: 1fr; }
  .app-topbar { padding: 0 1rem; }
}
```

4. Verification Screenshot: Responsive Layout & About Credentials

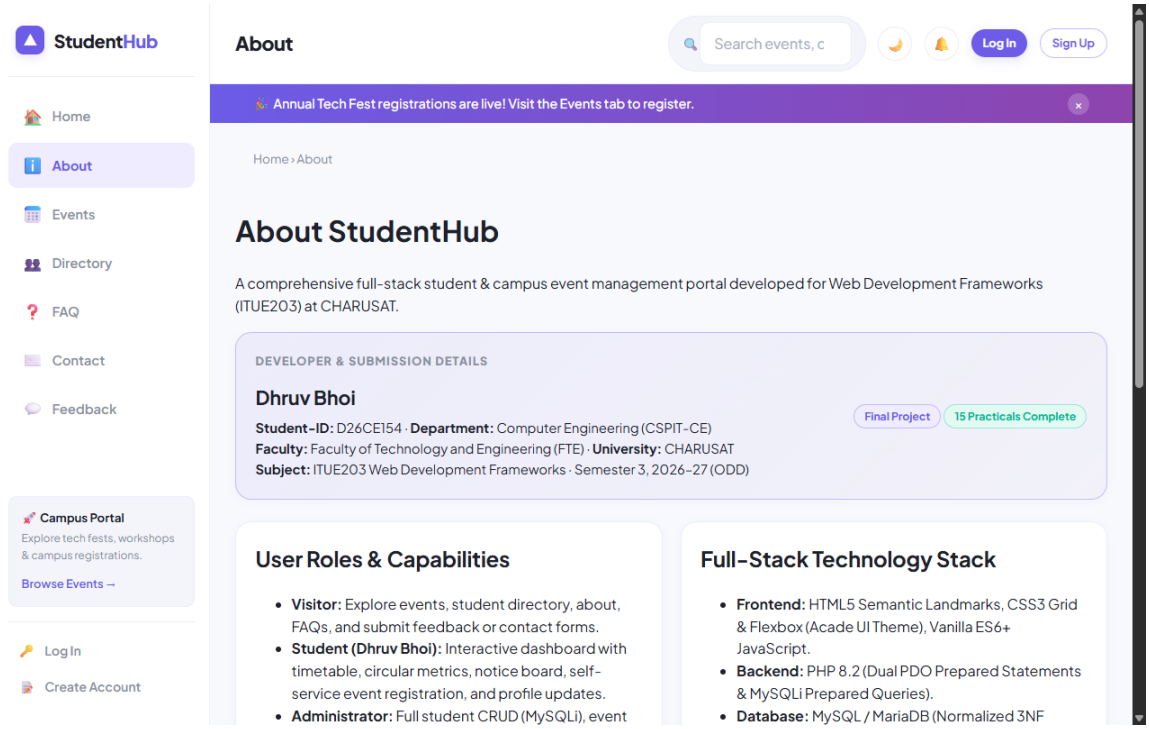


Figure: Modern Acade UI Shell & Student Developer Credentials Card (Dhruv Bhoi)

5. Key Questions & Analysis

Q: How does the layout adapt for mobile, tablet, and desktop screens?

Ans: Using a mobile-first responsive approach. Desktop screens display the persistent left navigation sidebar and multi-column grid cards. Tablet and mobile devices collapse the sidebar into an accessible slide-out drawer, transform grid layouts into single-column cards, and enable smooth touch scrolling.

Q: Which layout technique is used and why?

Ans: A hybrid approach: CSS Flexbox is used for 1-dimensional components (topbar headers, button groups, breadcrumbs, nav pills), while CSS Grid is used for 2-dimensional structures (dashboard timetable rows, event catalogs, stats cards, and multi-field forms).

Practical 4: JavaScript DOM Manipulation, Event Handling, and UI Interactivity

Practical Aim & Tools
Aim: Implement UI interactivity, DOM manipulation, event handling, dark/light theme switcher with localStorage, accordion FAQ, modal popups, banner notifications, and carousel slider.

Tools & Technologies: Vanilla ES6+ JavaScript, DOM APIs, Event Delegation, HTML5 LocalStorage

1. DOM Interactivity Modules

StudentHub features lightweight, dependency-free Vanilla ES6+ JavaScript (`assets/js/main.js`) powering five key UI features:
1. Theme Switcher: Reads and writes user preference (`light` or `dark`) to `localStorage`. An inline IIFE prevents theme flash on page load.
2. Collapsible Accordion: Implemented in `faq.php` with keyboard accessibility and ARIA expanded state management.
3. Accessible Modal Dialog: Accessible trap focus dialog for Quick Tours and Registration previews.
4. Sliding Highlights Carousel: Automated and keyboard-accessible slide transitions on the homepage.
5. Dismissible Status Banner: Persistent notification banner with smooth dismissal animations.

2. Implementation Code: Theme Toggle & Accordion

Listing: main.js — Theme Switcher and Accessible Accordion

```
// Theme Toggle with LocalStorage Persistence
const themeToggle = document.getElementById('themeToggle');
themeToggle?.addEventListener('click', () => {
  const current = document.documentElement.getAttribute('data-theme') || 'light';
  const target = current === 'dark' ? 'light' : 'dark';
  document.documentElement.setAttribute('data-theme', target);
  localStorage.setItem('sh_theme', target);
  themeToggle.textContent = target === 'dark' ? '🌞' : '🌙';
});

// Accessible FAQ Accordion
document.querySelectorAll('.accordion-header').forEach(btn => {
  btn.addEventListener('click', () => {
    const expanded = btn.getAttribute('aria-expanded') === 'true';
    btn.setAttribute('aria-expanded', !expanded);
    const body = btn.nextElementSibling;
    body.hidden = expanded;
  });
});
```

3. Verification Screenshot: Interactive FAQ Accordion

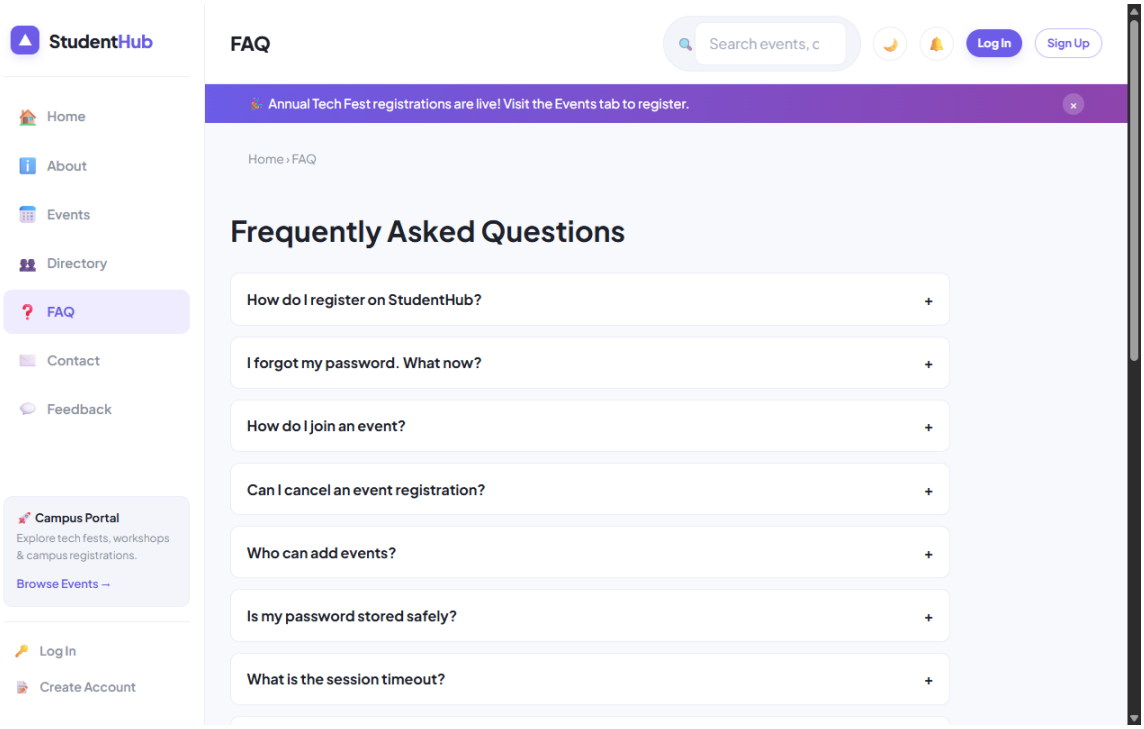


Figure: Interactive FAQ Accordion with Search and Dark/Light Mode Support

4. Key Questions & Analysis

Q: How are DOM elements selected and modified?

Ans: Elements are selected using modern `document.querySelector()` and `document.querySelectorAll()`. Modifications use secure DOM APIs like `element.textContent`, `element.setAttribute()`, and `element.classList.toggle()` to prevent cross-site scripting (XSS) risks.

Q: Is localStorage used for remembering user theme preferences?

Ans: Yes. When the theme toggle button is clicked, `localStorage.setItem('sh_theme', target)` persists the state. An immediate synchronous inline script in `<head>` executes before the DOM renders to prevent visual flickering.

Practical 5: Registration Form with Frontend Validation and User-Friendly Error Handling

Practical Aim & Tools

Aim: Build a responsive user registration form with comprehensive frontend validation, real-time error feedback, regex patterns, password strength meter, and Canvas CAPTCHA.

Tools & Technologies: HTML5 Form Validation, JavaScript Regex, Canvas API, Real-time DOM Feedback

1. Frontend Validation Rules & Pattern Specifications

• Full Name: RegEx `/^[A-Za-z][A-Za-z .'-]{2,59}$/` ensuring alphabetic letters, length between 3 and 60 characters.
• Email: Standard RFC email regex with domain check.
• Mobile: Indian phone number standard `/^[6-9]\d{9}$/` enforcing exactly 10 digits starting with 6, 7, 8, or 9.
• Password Complexity: Enforces minimum 8 characters, at least one uppercase letter, one lowercase letter, one digit, and one special character.
• Dynamic Strength Meter: Calculates entropy score (0 to 4) and dynamically renders color-coded progress bars (Weak, Moderate, Strong).
• Procedural Canvas CAPTCHA: Renders 5 randomized alphanumeric characters on an HTML5 `<canvas>` element with noise lines to deter automated bots.

2. Implementation Code: Password Strength & Canvas CAPTCHA

Listing: Procedural Canvas CAPTCHA Generation

```
// Canvas CAPTCHA Generation
function generateCaptcha() {
  const canvas = document.getElementById('captcha');
  if (!canvas) return;
  const ctx = canvas.getContext('2d');
  const chars = '23456789ABCDEFGHJKLMNPQRSTUVWXYZ';
  let code = '';
  for (let i = 0; i < 5; i++) code += chars.charAt(Math.floor(Math.random() * chars.length));
  window.captchaCode = code;
  ctx.fillStyle = '#EEECFE';
  ctx.fillRect(0, 0, canvas.width, canvas.height);
  // Add noise & draw distorted text
  ctx.font = 'bold 22px Courier New';
  ctx.fillStyle = '#6C5CE7';
  ctx.fillText(code, 20, 32);
}
```

3. Verification Screenshot: Registration Form

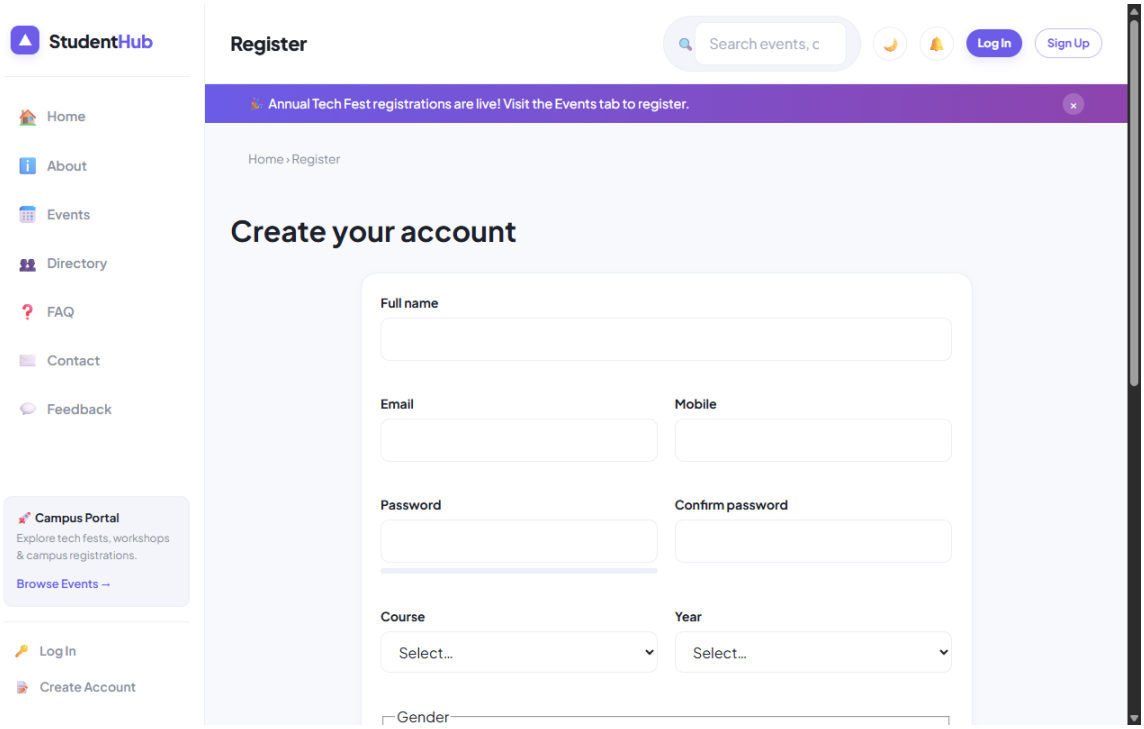


Figure: Registration Form with Real-Time Validation, Password Meter and Canvas CAPTCHA

4. Key Questions & Analysis

Q: Are correct input types and validation attributes used?

Ans: Yes. Appropriate HTML5 input types (`type="email"`, `type="tel"`, `type="password"`) are coupled with `required`, `autocomplete`, `inputmode="numeric"`, and `novalidate` on the form to allow JavaScript custom validation messages.

Q: Is password strength checked and displayed dynamically?

Ans: Yes. An `input` event listener evaluates password length and character diversity, updating a `<div id="strengthBar">` with visual width and color cues (Red = Weak, Orange = Moderate, Purple/Green = Strong).

Practical 6: Rendering External JSON Data using Fetch API, Search, and Filter

Practical Aim & Tools

Aim: Fetch and render external JSON datasets using the Fetch API, async/await, debounced search, category filters, dependent State-City dropdowns, sorting, and client-side pagination.

Tools & Technologies: JavaScript Fetch API, Promises, Async/Await, JSON Parsing, Debounce Utility

1. Architecture: JSON Data Rendering (`assets/js/data-render.js`)

StudentHub features two dynamic catalog modules backed by JSON files:

• Events Catalog (`events.php` / `data/events.json`): Fetches campus hackathons, seminars, and sports meets. Supports real-time text search, category filtering (Workshop, Technical, Cultural, Sports), date/title sorting, and pagination (6 items per page).

• Student Directory (`directory.php` / `data/students.json`): Fetches student directory records. Features a dependent State → City filter (selecting Gujarat dynamically populates Anand, Vadodara, Ahmedabad, Surat) along with course filters and sorting.

2. Implementation Code: Fetch API & Dependent Dropdowns

Listing: data-render.js — Async Fetch and Dependent Filter Logic

```
async function loadData(src) {
  statusEl.textContent = 'Loading records…';
  try {
    const res = await fetch(src);
    if (!res.ok) throw new Error('HTTP ' + res.status);
    allData = await res.json();
    statusEl.textContent = '';
    render();
  } catch (err) {
    statusEl.innerHTML = '<span class="error">Failed to load records.</span>';
  }
}

// Dependent State -> City Filter
const CITIES = {
  'Gujarat': ['Anand', 'Vadodara', 'Ahmedabad', 'Surat', 'Rajkot'],
  'Maharashtra': ['Mumbai', 'Pune', 'Nagpur', 'Nashik']
};
stateSelect?.addEventListener('change', () => {
  const st = stateSelect.value;
  citySelect.innerHTML = '<option value="">All cities</option>';
  if (CITIES[st]) {
    citySelect.disabled = false;
    CITIES[st].forEach(c => citySelect.add(new Option(c, c)));
  } else {
    citySelect.disabled = true;
  }
  render();
});
```

3. Verification Screenshot: Dynamic Events Catalog

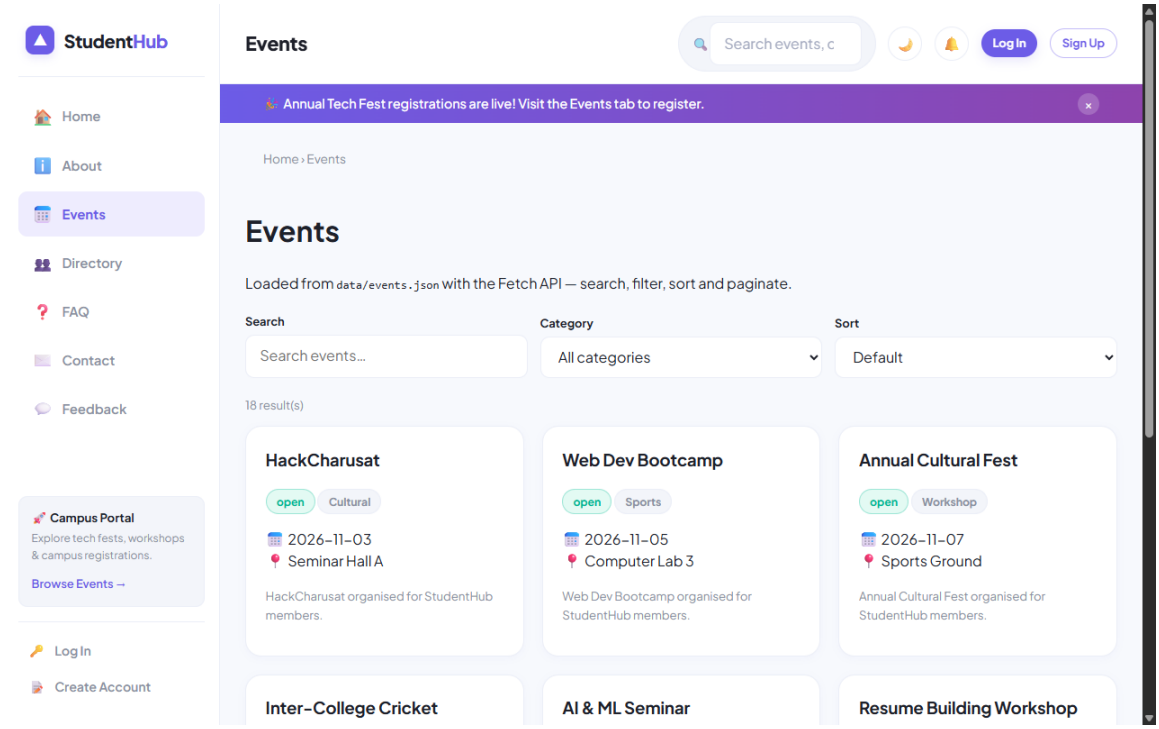


Figure: Events Catalog with Debounced Search, Category Filter, and Dynamic Cards Grid

4. Verification Screenshot: Student Directory with Dependent Dropdown

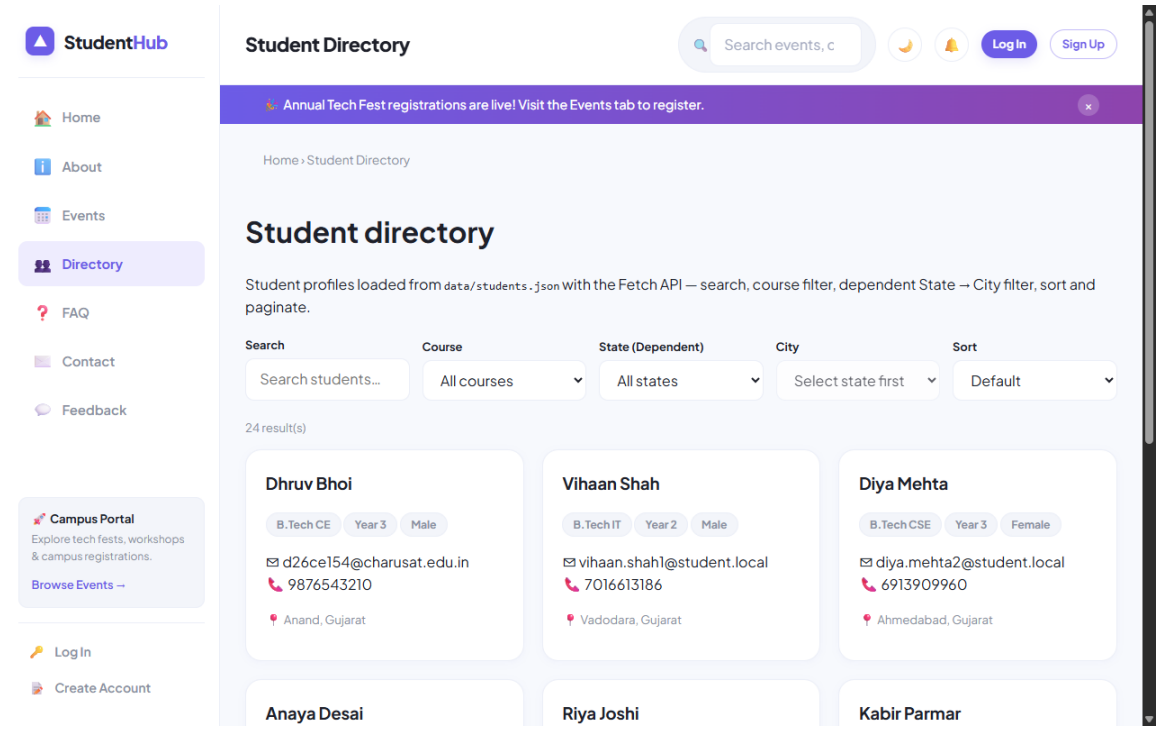


Figure: Student Directory with Dependent State -> City Filter and Top Student Dhruv Bhoi

5. Key Questions & Analysis

Q: How is JSON fetched, parsed, and rendered dynamically?

Ans: The browser calls `window.fetch(src)`, returns a Promise that resolves to a Response object, and parses the payload with `res.json()`. The resulting JavaScript array is transformed through `.filter()`, `.sort()`, and `.slice()` before building DOM elements via template literals.

Q: How are loading, error, and empty states handled?

Ans: A dedicated `<div id="dataStatus">` element with `aria-live="polite"` shows a spinner during network transport, displays user-friendly retry alerts if `!res.ok`, and shows an 'Empty records found' notice if the search query matches 0 records.

Practical 7: PHP Form Processing with Server-Side Validation and CSV/JSON File Storage

Practical Aim & Tools
Aim: Implement PHP form processing for contact and feedback submissions with server-side validation, data sanitization, CSRF protection, and persistent storage in JSON/CSV files.

Tools & Technologies: PHP 8.x, CSV File I/O (`fputcsv`), JSON Storage (`json_encode`), CSRF Protection

1. Server-Side Processing Architecture

Client-side validation is easily bypassed; therefore, robust server-side processing is mandatory for security. StudentHub implements two storage workflows:
1. Contact Us (`contact.php`): Validates user inquiries, strips HTML tags (`clean()`), checks email formats (`FILTER_VALIDATE_EMAIL`), verifies anti-CSRF session tokens, and appends records atomically to `data/contacts.csv` using `fputcsv()` with exclusive file locks (`LOCK_EX`).
2. Student Feedback (`feedback.php`): Collects user star ratings (1 to 5) and comments, validates data integrity, and stores entries in `data/feedback.json`. Recent feedback cards are dynamically read and rendered below the form.

2. Implementation Code: Contact Processing & Atomic CSV Write

Listing: contact.php — Input Sanitization, CSRF Verification, and Atomic File Append

```
// contact.php - Server-Side Processing
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!csrf_check()) {
        $errors['form'] = 'Security validation failed (Invalid CSRF token).';
    } else {
        $name = clean($_POST['name'] ?? '');
        $email = clean($_POST['email'] ?? '');
        $message = clean($_POST['message'] ?? '');

        if (strlen($name) < 2) $errors['name'] = 'Name must be at least 2 characters.';
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) $errors['email'] = 'Invalid email
address.';
        if (strlen($message) < 10) $errors['message'] = 'Message must be at least 10
characters.';

        if (!$errors) {
            $fp = fopen(BASE_PATH . '/data/contacts.csv', 'a');
            if ($fp && flock($fp, LOCK_EX)) {
                fputcsv($fp, [date('Y-m-d H:i:s'), $name, $email, $message,
$_SERVER['REMOTE_ADDR']]);
                flock($fp, LOCK_UN);
                fclose($fp);
                flash('success', 'Thank you! Your message has been recorded.');
                redirect('contact.php');
            }
        }
    }
```

```
}
```

3. Verification Screenshot: Feedback & Rating System

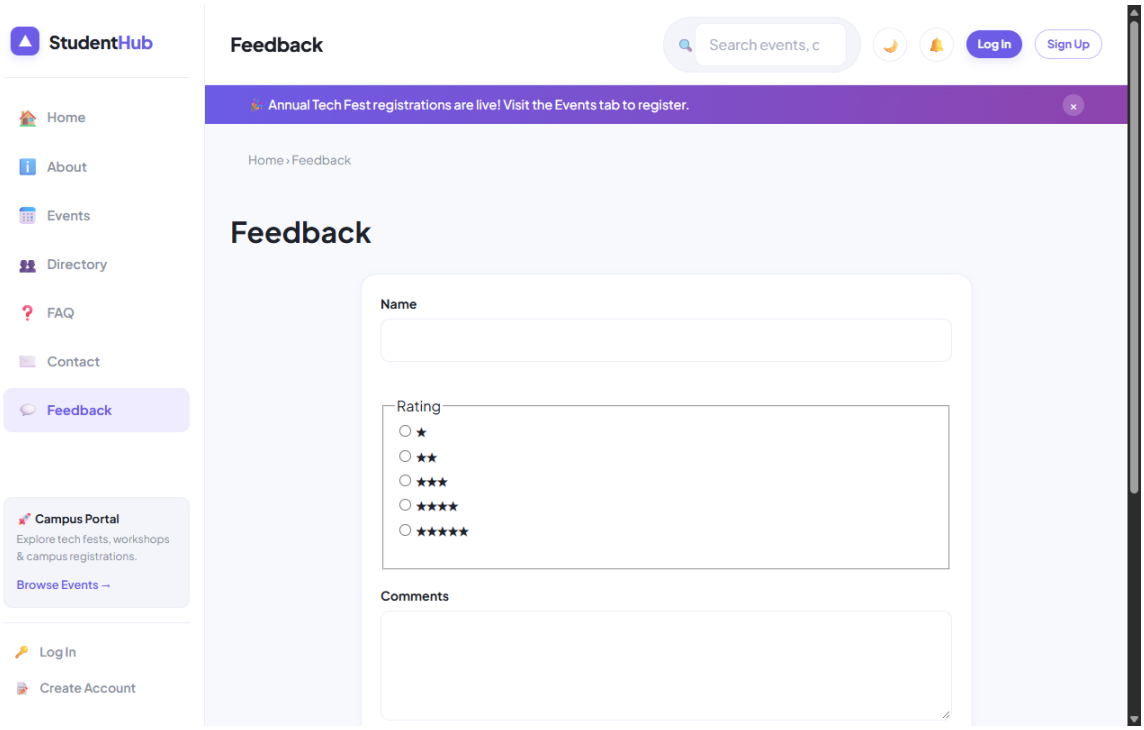


Figure: Feedback Submission Form with Star Ratings and Persisted JSON Feed

4. Verification Screenshot: Contact Inquiries Portal

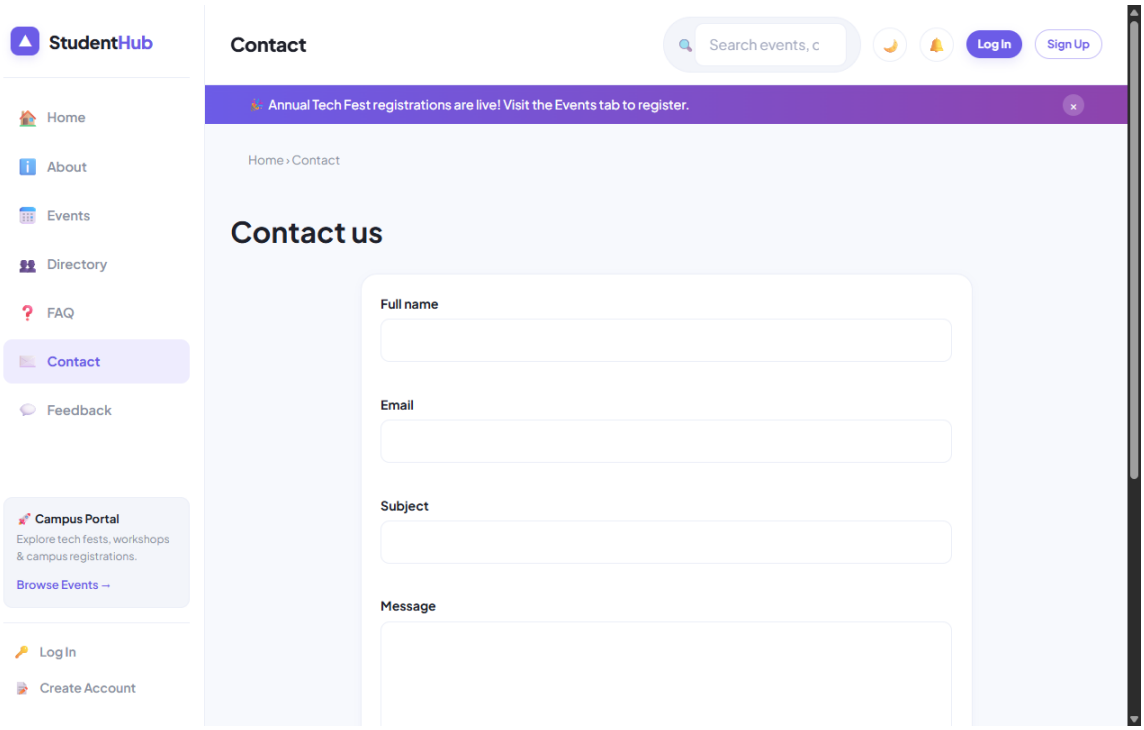


Figure: Contact Inquiries Form with Server-Side Validation and CSV Audit Logging

5. Key Questions & Analysis

Q: Why must form data be submitted using POST instead of GET?

Ans: GET appends parameters to the query string in browser history, server logs, and HTTP referrers, making it insecure for sensitive data. POST transmits data inside the HTTP message body, supports unlimited payload sizes, and complies with HTTP idempotency standards.

Q: How is file writing handled safely against concurrent race conditions?

Ans: By invoking `flock($fp, LOCK_EX)` before performing file writes. Exclusive locking prevents data corruption when multiple users submit forms simultaneously.

Practical 8: MySQL Schema Design, ER Model, Database Connectivity, and Diagnostics

Practical Aim & Tools
Aim: Design a normalized (3NF) relational database schema for StudentHub, construct an ER model, establish PDO and MySQLi connections, and implement a diagnostic connection test suite.

Tools & Technologies: MySQL 8.x, InnoDB Engine, Foreign Key Constraints, PDO, MySQLi Prepared Statements

1. Entity-Relationship Model & 3NF Normalization

The StudentHub database (`studenthub`) is designed in Third Normal Form (3NF) to eliminate data redundancy and ensure relational integrity:
• `users`: Authentication credentials, roles (admin/student), password hashes, and activity status.
• `students`: Academic profiles (course, year, contact, student ID) bound to `users.id` with a 1-to-1 foreign key and `ON DELETE CASCADE`.
• `events`: Campus events, dates, venues, capacities, and poster image filenames.
• `registrations`: Student-to-event registrations (Many-to-Many mapping) with a `UNIQUE KEY (event_id, student_id)` preventing duplicate registrations.
• `remember_tokens`: Cryptographic selectors and SHA-256 token hashes for persistent 'Remember Me' sessions.
• `audit_logs`: Immutable security audit trail recording user logins, updates, IP addresses, and timestamps.
• Stored Procedures: `sp_registrations_by_month` (analytics grouping) and `sp_event_registration_count` (dynamic count aggregation).

2. Implementation Code: Dual Database Driver (`includes/db.php`)

Listing: includes/db.php — UTF-8MB4 Dual PDO and MySQLi Initialization

```
// includes/db.php - Secure Dual Database Driver
define('DB_HOST', 'localhost');
define('DB_NAME', 'studenthub');
define('DB_USER', 'root');
define('DB_PASS', '');

// 1. PDO Driver (Object-Oriented, Exception-driven)
$dsn = 'mysql:host=' . DB_HOST . ';dbname=' . DB_NAME . ';charset=utf8mb4';
$opt = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES   => false,
];
$pdo = new PDO($dsn, DB_USER, DB_PASS, $opt);

// 2. MySQLi Driver (For explicit prepared statements & multi-queries)
$mysqli = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
$mysqli->set_charset('utf8mb4');
```

3. Verification Screenshot: Comprehensive DB Diagnostic Test Suite

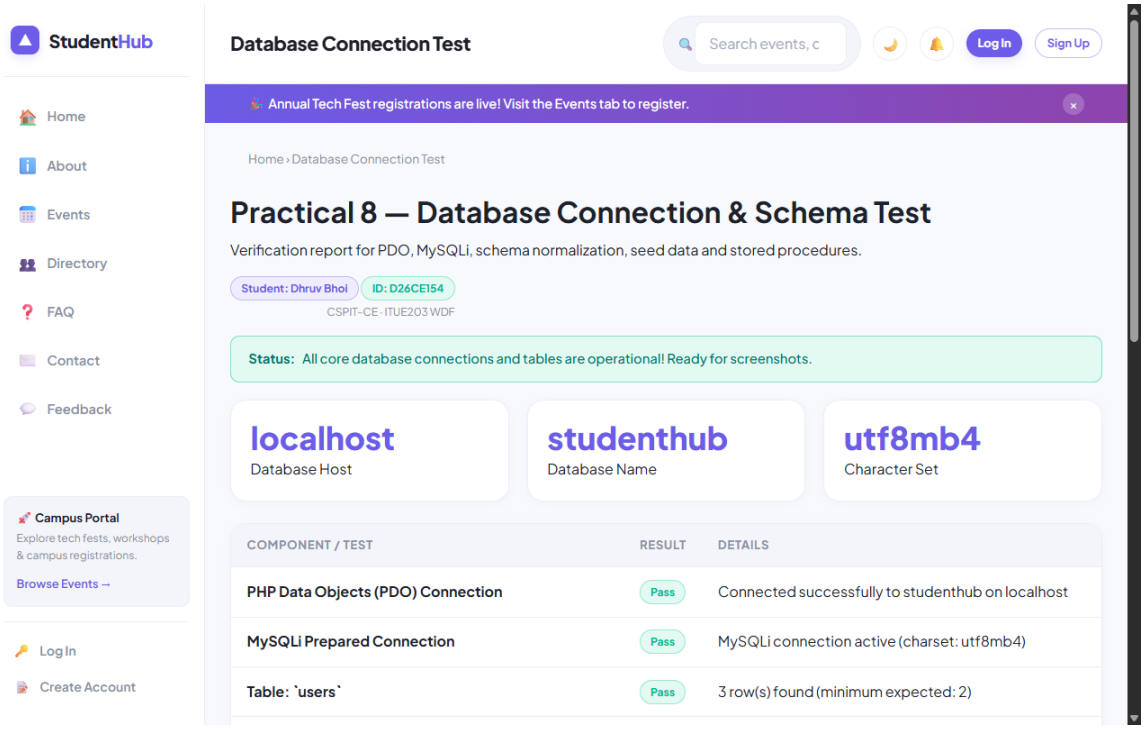


Figure: Live Diagnostic Test Suite (db-test.php) Demonstrating 6/6 PASS for Dhruv Bhoi

4. Key Questions & Analysis

Q: Is the schema normalized properly into 3NF?

Ans: Yes. 1NF is satisfied (all attributes are atomic; no repeating groups). 2NF is satisfied (no partial dependencies on composite keys). 3NF is satisfied (no transitive dependencies; non-key attributes depend solely on the primary key).

Q: Why is `ATTR_EMULATE_PREPARES => false` specified in PDO?

Ans: Native prepared statements ensure that SQL statement parsing and user data binding happen in separate network packets inside MySQL server. This completely eliminates SQL Injection vulnerabilities regardless of character encodings.

Practical 9: Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

Practical Aim & Tools
Aim: Build secure user registration with server-side validation, duplicate email check, bcrypt password hashing (`password_hash`), database transactions, and audit logging.

Tools & Technologies: PHP 8.x, MySQLi Prepared Statements, Bcrypt Algorithm, ACID Database Transactions

1. Secure Registration Workflow & Threat Mitigation

User registration (`register.php`) establishes student identities securely through five defensive layers:
1. Server Validation: Re-executes all pattern validations independently of frontend checks.
2. Duplicate Check: Executes a parameterized SELECT query to verify email uniqueness before attempting insertion.
3. Cryptographic Hashing: Generates secure Bcrypt salted hashes using `password_hash($password, PASSWORD_DEFAULT)` with default cost factor 10.
4. Database Transactions: Wraps `users` insert and `students` insert within `$mysqli->begin_transaction()`. If either fails, `$mysqli->rollback()` ensures zero orphaned records.
5. Security Audit Log: Records registration event, user ID, client IP address, and timestamp into `audit_logs`.

2. Implementation Code: Transactional Registration

Listing: register.php — Atomic ACID Registration with Bcrypt Hashing

```
// register.php - Transactional Registration
$hash = password_hash($d['password'], PASSWORD_DEFAULT);
$mysqli->begin_transaction();
try {
    $u = $mysqli->prepare('INSERT INTO users (name, email, password_hash, role) VALUES (?, ?, ?, ?)');
    $role = 'student';
    $u->bind_param('ssss', $old['name'], $d['email'], $hash, $role);
    $u->execute();
    $uid = $mysqli->insert_id;

    $s = $mysqli->prepare('INSERT INTO students (user_id, name, email, mobile, course, year, gender) VALUES (?, ?, ?, ?, ?, ?, ?)');
    $s->bind_param('issssis', $uid, $old['name'], $d['email'], $d['mobile'], $d['course'], $d['year'], $d['gender']);
    $s->execute();

    $mysqli->commit();
    flash('success', 'Registration successful! You may now log in.');
    redirect('login.php');
} catch (Exception $e) {
    $mysqli->rollback();
    $errors['form'] = 'Registration failed. Please try again.';
```

```
}
```

3. Verification Screenshot: Registration Form & Error States

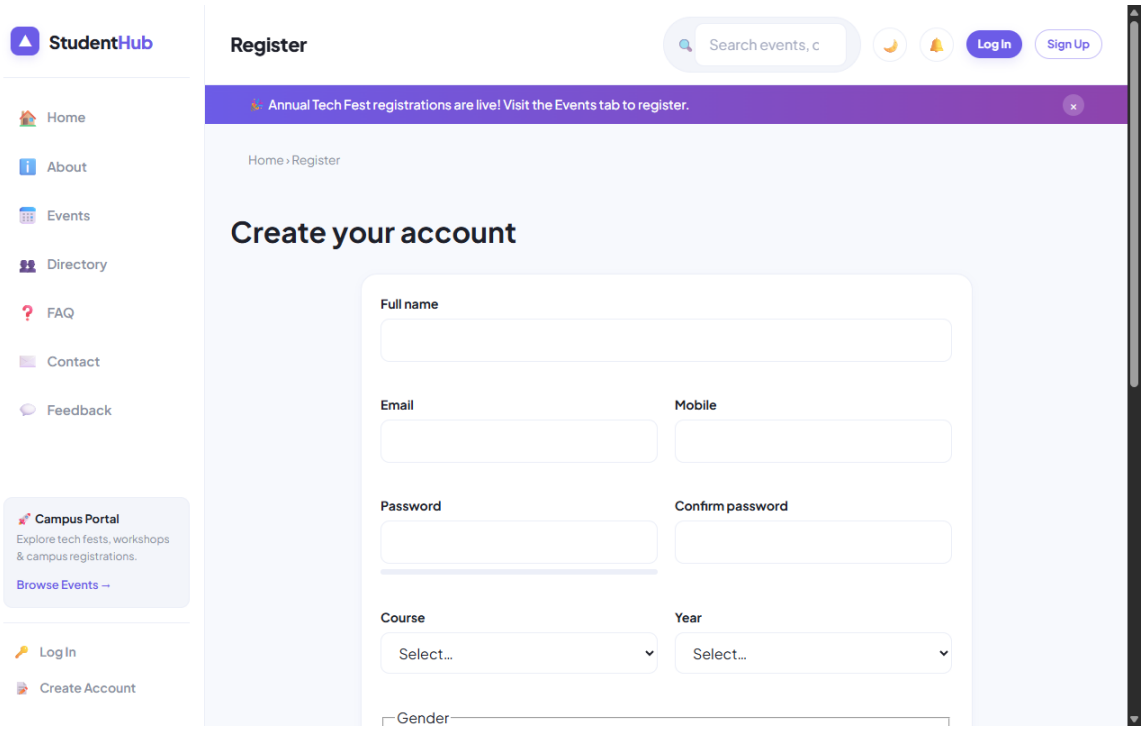


Figure: Registration Form Demonstrating Validation Cues and Account Creation

4. Key Questions & Analysis

Q: Why should MD5 or SHA1 never be used for password storage?

Ans: MD5 and SHA-1 are high-speed general hashing algorithms that lack computational work factors. Modern GPUs compute billions of MD5 hashes per second, making rainbow table and brute-force attacks trivial. Bcrypt incorporates an adaptive slow cost factor and automatic random salt generation.

Q: How does the system prevent duplicate account registrations?

Ans: Through a two-step defense: 1) A parameterized SQL query verifies if the email exists prior to hashing, and 2) A database-level `UNIQUE KEY (email)` constraint on the `users` table acts as a failsafe, catching race conditions via SQL exception code 1062.

Practical 10: Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Practical Aim & Tools
Aim: Implement secure authentication (login/logout) using PHP sessions, `password_verify()`, session regeneration against fixation, role-based access control (RBAC), 15-minute inactivity timeout, and secure remember-me cookies.

Tools & Technologies: PHP Sessions, Cryptographic Tokens, HTTPOnly/Secure Cookies, RBAC Architecture

1. Session Management & Defense-in-Depth Authentication

StudentHub (`login.php`, `logout.php`, `includes/functions.php`) enforces enterprise security standards:
• Secure Authentication: Uses `password_verify($password, $hash)` resisting timing attacks.
• Session Fixation Defense: Calls `session_regenerate_id(true)` upon successful authentication to destroy the pre-login session ID.
• Inactivity Timeout: Automatically tracks `$_SESSION['last_activity']`. Sessions exceeding 15 minutes (`SESSION_TIMEOUT = 900`) are destroyed with a timeout redirect.
• Split-Token Remember-Me: Stores a public 12-byte hex selector and a private 32-byte token hash in `remember_tokens`. Cookies are marked `HttpOnly`, `SameSite=Lax`, and `Expires = 14 days`.
• Role-Based Access Control (RBAC): `require_role('admin')` blocks students from administrative scripts (`students.php`, `manage_events.php`, `admin/`).

2. Implementation Code: Secure Login & Remember-Me

Listing: login.php — Authentication, Session Regeneration, and Split-Token Generation

```
// login.php - Authentication & Token Issue
$st = $pdo->prepare('SELECT * FROM users WHERE email = ? AND status = "active"');
$st->execute([$email]);
$u = $st->fetch();

if ($u && password_verify($pass, $u['password_hash'])) {
    session_regenerate_id(true);
    $_SESSION['user'] = ['id' => $u['id'], 'name' => $u['name'], 'email' => $u['email'], 'role'
=> $u['role']];
    $_SESSION['last_activity'] = time();
    $pdo->prepare('UPDATE users SET last_login = NOW() WHERE id = ?')->execute([$u['id']]);

    if (!empty($_POST['remember'])) {
        $sel = bin2hex(random_bytes(12));
        $tok = bin2hex(random_bytes(32));
        $pdo->prepare('INSERT INTO remember_tokens (user_id, selector, token_hash, expires_at)
VALUES (?, ?, ?, DATE_ADD(NOW(), INTERVAL 14 DAY))')
            ->execute([$u['id'], $sel, hash('sha256', $tok)]);
        setcookie('remember', "$sel:$tok", ['expires' => time() + 14*86400, 'path' => '/',
'httponly' => true, 'samesite' => 'Lax']);
    }
```

```
    redirect($u['role'] === 'admin' ? 'admin/dashboard.php' : 'dashboard.php');
}
```

3. Verification Screenshot: Login Portal & Demo Helper

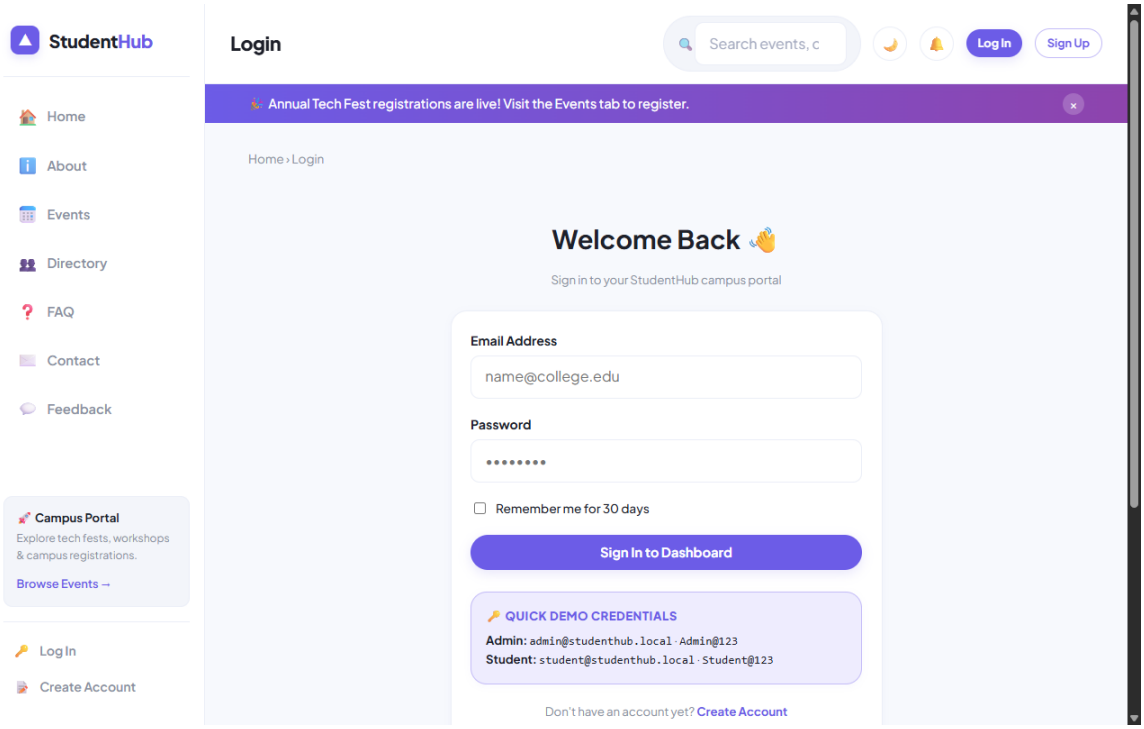


Figure: Secure Login Screen with Role Credentials Helper and Remember-Me Toggle

4. Verification Screenshot: Student Dashboard (Dhruv Bhoi)

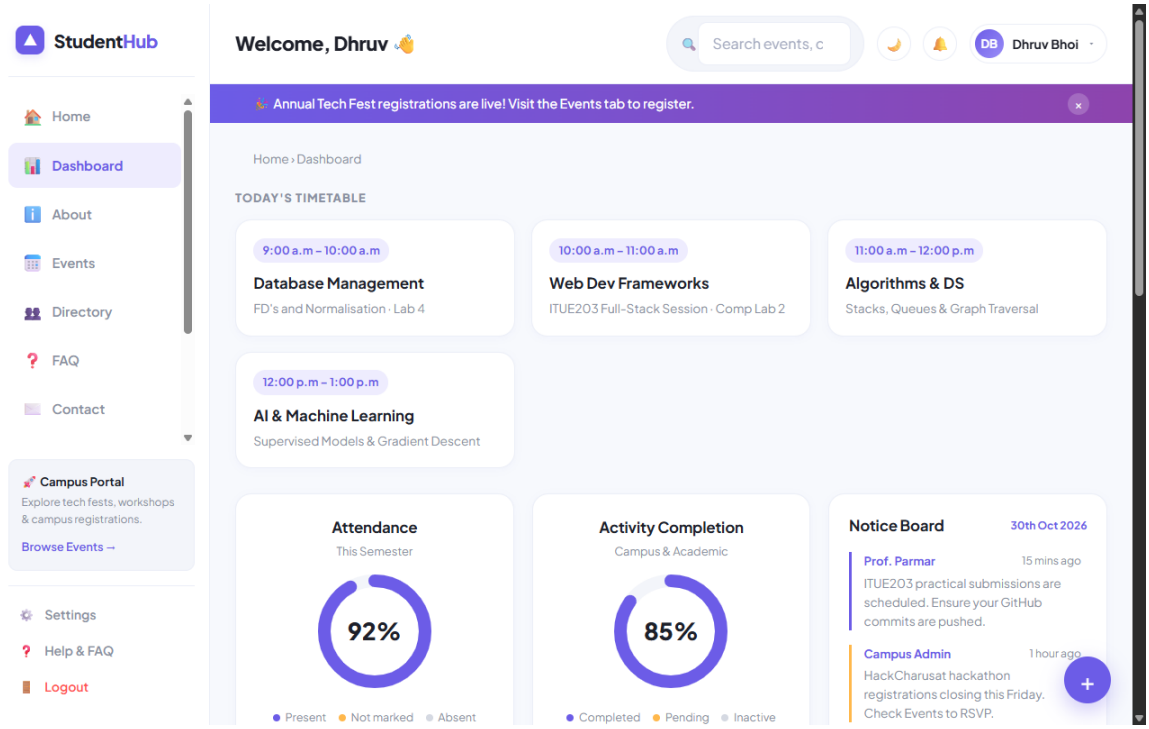


Figure: Authenticated Student Dashboard with Timetable, Progress Donuts, and Events Feed

5. Key Questions & Analysis

Q: How does `session_regenerate_id(true)` protect against session fixation attacks?

Ans: In a session fixation attack, an adversary tricks the victim into using a predetermined session ID. Calling `session_regenerate_id(true)` issues a brand new cryptographically random session identifier while deleting the old session storage, invalidating any fixation token held by the attacker.

Q: How does the split-token 'Remember Me' mechanism prevent database leakage attacks?

Ans: The database stores only the public selector and a SHA-256 hash of the validator token. If an attacker breaches the database, they cannot forge session cookies because the raw validator token is never stored server-side.

Practical 11: Student Management CRUD with Search, Filter, Pagination, Soft Delete, and CSV Export

Practical Aim & Tools
Aim: Develop complete Student Management CRUD operations for administrators with MySQLi prepared statements, search, course filter, pagination, soft delete/restore, and CSV export.

Tools & Technologies: PHP, MySQLi Prepared Statements, CSV Stream Generation, Soft Delete Architecture

1. Administrative CRUD Features (`students.php`)

StudentHub empowers administrators to oversee student records with enterprise-grade safeguards:
• Parameterized CRUD: All Create, Read, Update, and Delete operations use strict MySQLi prepared statements.
• Search & Course Filters: Instant lookup across student name, email, and mobile, coupled with academic course dropdowns.
• Pagination Algorithm: Efficient `LIMIT ?, ?` pagination with calculated offsets (`($page - 1) * $perPage`).
• Soft Delete & Restore: Rather than irrevocably purging database rows, deletions toggle `status = 'deleted'`. Administrators can toggle between 'Active' and 'Deleted' views to restore records.
• On-Demand CSV Export: Clicking 'Export CSV' streams database records directly via `fputcsv()` with HTTP headers `Content-Type: text/csv`.

2. Implementation Code: Prepared CRUD & Soft Delete

Listing: students.php — Prepared Statement Soft Delete and Audit Logging

```
// students.php - Soft Delete / Restore
if ($_SERVER['REQUEST_METHOD'] === 'POST' && in_array($_POST['do'], ['delete', 'restore'])) {
    if (!csrf_check()) flash('error', 'Security token invalid.');
    else {
        $id = (int)$_POST['id'];
        $status = $_POST['do'] === 'delete' ? 'deleted' : 'active';
        $st = $mysqli->prepare('UPDATE students SET status=? WHERE id=?');
        $st->bind_param('si', $status, $id);
        $st->execute();
        audit($pdo, 'student_' . $status, 'Student ID: ' . $id);
        flash('success', $status === 'deleted' ? 'Student moved to deleted (soft delete).' :
'Student restored successfully.');
    }
    redirect('students.php');
}
```

3. Verification Screenshot: Student Management CRUD Interface

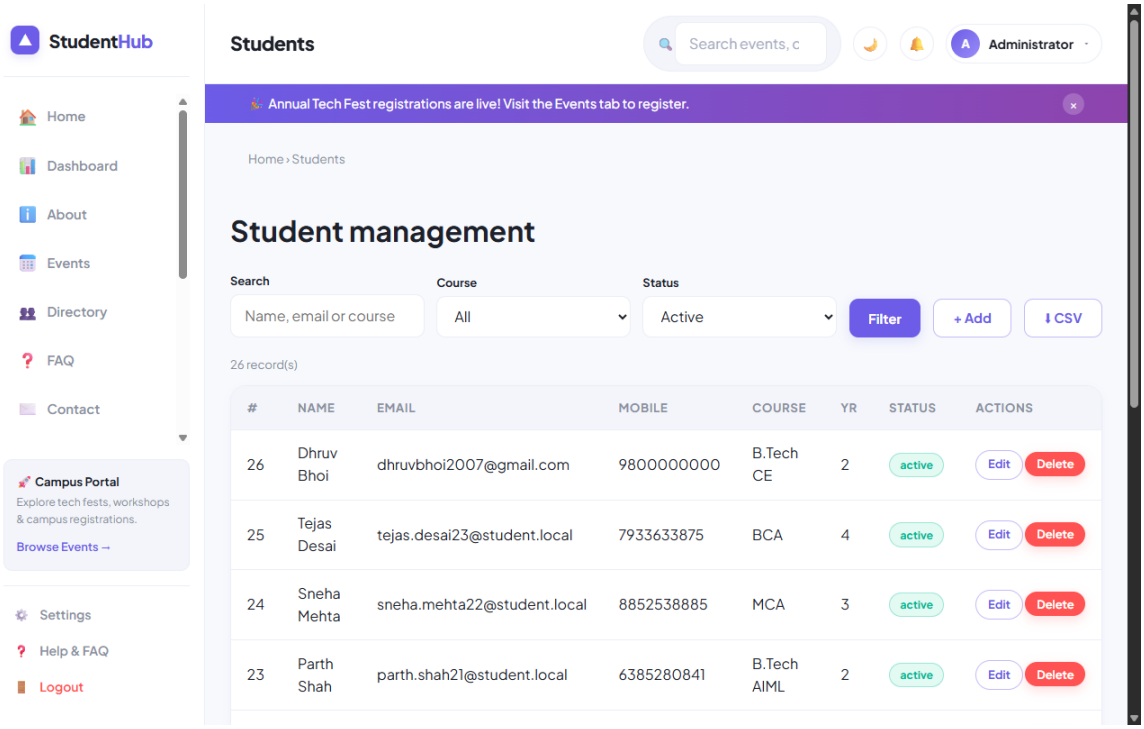


Figure: Student Management CRUD with Search Toolbar, Course Filter, Pagination, and Status Badges

4. Key Questions & Analysis

Q: Why is soft delete preferable over hard delete in enterprise applications?

Ans: Hard delete immediately destroys historical data, breaking foreign key relationships in audit logs, event registrations, and invoices. Soft delete preserves data integrity, prevents accidental data loss, complies with data retention laws, and allows 1-click recovery.

Q: How is CSV export streamed efficiently without exhausting server memory?

Ans: By opening the output stream wrapper `fopen('php://output', 'w')` and writing records iteratively with `fputcsv($out, $row)` directly to the HTTP client socket, buffering 0 intermediate memory.

Practical 12: Event Management CRUD with Poster Upload and File Validation

Practical Aim & Tools
Aim: Build an Event Management CRUD interface with poster image upload, MIME type and size validation, secure unique filename generation, directory permissions, and database tracking.

Tools & Technologies: PHP File Upload Handling (`$_FILES`), MIME Type Detection (`finfo`), File Security

1. Secure File Upload Architecture (`manage_events.php`)

File uploading is one of the most common web attack vectors (Remote Code Execution, Malicious Script Injection). StudentHub defends poster uploads through 5 strict verification checks:
1. HTTP POST & Error Check: Verifies `is_uploaded_file()` and checks `$_FILES['poster']['error'] === UPLOAD_ERR_OK`.
2. File Size Cap: Restricts files to a strict 2 Megabyte ceiling (`2 * 1024 * 1024` bytes).
3. MIME Content Inspection: Uses PHP's `finfo(FILEINFO_MIME_TYPE)` on temporary file bytes, permitting only `image/jpeg`, `image/png`, and `image/webp`. Never relies on client-supplied extensions.
4. Cryptographic Renaming: Files are renamed using `bin2hex(random_bytes(16)) . '.' . $ext` to prevent path traversal and execution.
5. Directory Execution Prevention: The `uploads/` folder contains an `index.php` and an `.htaccess` disabling script execution.

2. Implementation Code: Poster Upload & MIME Inspection

Listing: manage_events.php — MIME Type Verification and Cryptographic Renaming

```
// manage_events.php - Poster Validation
if (!empty($_FILES['poster']['name'])) {
    $f = $_FILES['poster'];
    if ($f['size'] > 2 * 1024 * 1024) $errors['poster'] = 'Poster must be under 2 MB.';
    else {
        $finfo = new finfo(FILEINFO_MIME_TYPE);
        $mime = $finfo->file($f['tmp_name']);
        $map = ['image/jpeg'=>'jpg', 'image/png'=>'png', 'image/webp'=>'webp'];
        if (!isset($map[$mime])) {
            $errors['poster'] = 'Invalid format. Only JPG, PNG, and WEBP allowed.';
        } else {
            $filename = bin2hex(random_bytes(16)) . '.' . $map[$mime];
            move_uploaded_file($f['tmp_name'], BASE_PATH . '/uploads/' . $filename);
            $posterPath = 'uploads/' . $filename;
        }
    }
}
```

3. Verification Screenshot: Event Management CRUD

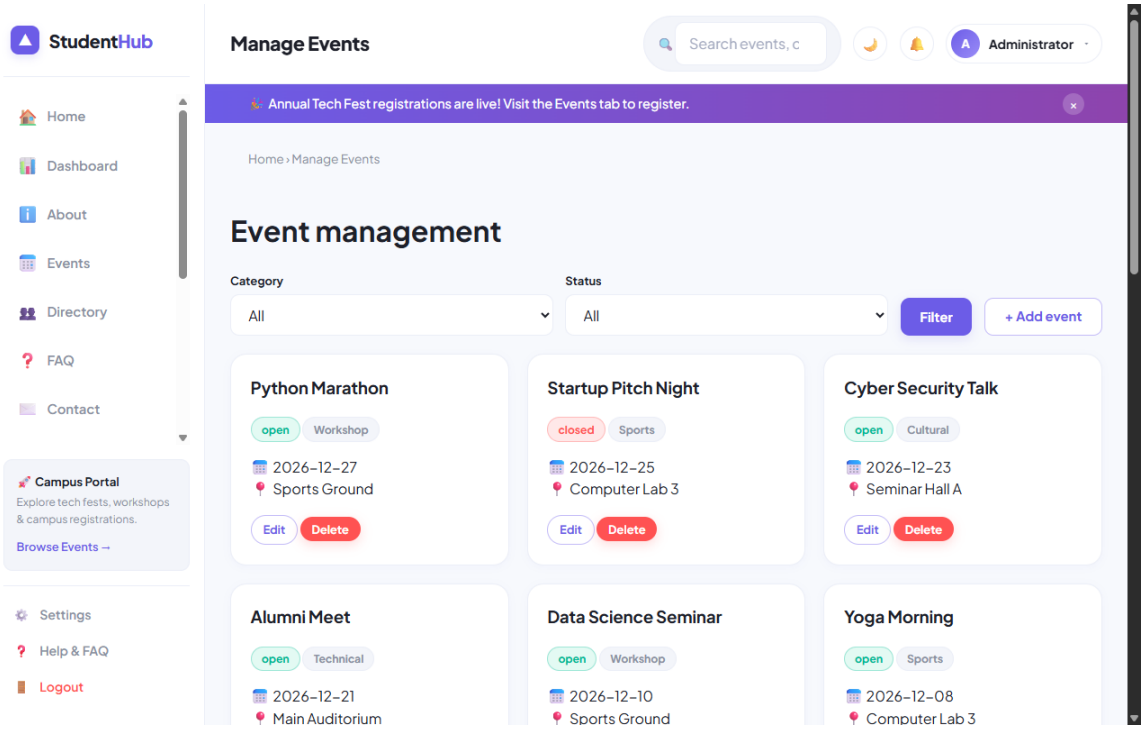


Figure: Event Management CRUD Interface with Poster Upload Field and Live Status Controls

4. Key Questions & Analysis

Q: Why should `$_FILES['poster']['type']` never be trusted?

Ans: `$_FILES['...']['type']` is a client-provided HTTP header that can be spoofed trivially using Postman or Burp Suite. A malicious user could upload a PHP shell named `shell.php` with header `Content-Type: image/jpeg`. Server-side verification must use `finfo_file` to read file magic bytes directly.

Q: How does cryptographic file renaming mitigate path traversal?

Ans: Renaming files to random hashes (e.g. `a3f8c...jpg`) strips dangerous characters like `../` or null bytes, preventing attackers from overwriting existing system files or guessing uploaded filenames.

Practical 13: PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Practical Aim & Tools
Aim: Create a RESTful PHP JSON API endpoint (`api/events.php`) supporting GET and POST, and integrate AJAX/Fetch CRUD operations without page reloads, including loading spinners and toast notifications.

Tools & Technologies: PHP REST API, JSON Serialization, AJAX, Fetch API, DOM Replacement

1. RESTful JSON API Architecture (`api/events.php`)

StudentHub implements a dedicated headless API endpoint adhering to REST conventions:
• Content Negotiation: Emits `Content-Type: application/json; charset=utf-8` on every response.
• HTTP Verbs & Routing: GET requests query events with optional debounced search filters; POST requests handle authenticated CRUD actions (create, delete, toggle status).
• Standard HTTP Status Codes: Returns `200 OK` on success, `401 Unauthorized` for guests, `403 Forbidden` for invalid CSRF tokens, `422 Unprocessable Entity` for validation errors, and `500 Internal Error` for failures.
• Client-Side Controller (`assets/js/api-crud.js`): Intercepts form submissions, transmits JSON payloads with `X-CSRF-Token` headers, and smoothly updates the DOM table without full page refreshes.

2. Implementation Code: API Router & AJAX Client

Listing: api/events.php — Parameterized REST API with HTTP Status Emission

```
// api/events.php - JSON API Router
header('Content-Type: application/json; charset=utf-8');
require_once __DIR__ . '/../includes/functions.php';
require_once __DIR__ . '/../includes/db.php';
require_login(); // Requires authenticated session (401 response on failure)

if ($_SERVER['REQUEST_METHOD'] === 'GET') {
    $q = clean($_GET['q'] ?? '');
    $sql = 'SELECT id, title, category, event_date, venue, status FROM events';
    if ($q !== '') $sql .= ' WHERE title LIKE ? OR venue LIKE ?';
    $st = $pdo->prepare($sql);
    $q !== '' ? $st->execute(["%$q%", "%$q%"]) : $st->execute();
    echo json_encode(['success' => true, 'data' => $st->fetchAll()]);
    exit;
}
```

3. Verification Screenshot: Live AJAX Events Interface

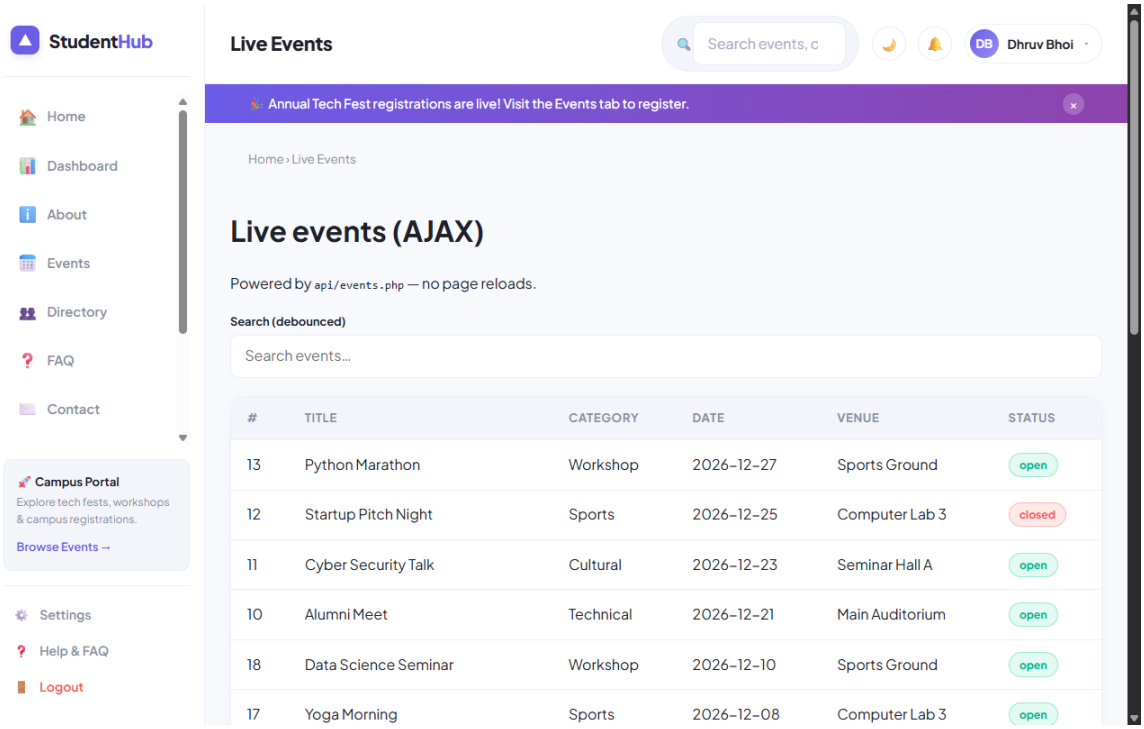


Figure: Live AJAX Events Dashboard (api-demo.php) Rendering Real-Time Updates without Reloads

4. Key Questions & Analysis

Q: How are CSRF tokens transmitted during asynchronous AJAX requests?

Ans: During page generation, the server places a CSRF token in a data attribute (`data-csrf="..."`). The JavaScript controller attaches this token to the custom request header `X-CSRF-Token`, which PHP validates via `csrf_check($_SERVER['HTTP_X_CSRF_TOKEN'])`.

Q: What are the benefits of Single Page Application (SPA) AJAX updates over traditional server-rendered pages?

Ans: AJAX updates significantly reduce network bandwidth by exchanging lightweight JSON rather than full HTML documents. The user experience is seamless without screen flashing, form inputs preserve focus, and responsiveness feels like a native desktop app.

Practical 14: Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log

Practical Aim & Tools

Aim: Develop an Admin Dashboard with role-based access control, KPI metric cards, dynamic navigation, stored procedure monthly registration charts, user active/inactive toggles, and an audit trail viewer.

Tools & Technologies: PHP, MySQL Stored Procedures, Role-Based Access Control (RBAC), CSS Data Visualization

1. Enterprise Admin Features (`admin/dashboard.php` & `admin/audit.php`)

The administrative module provides executive-level monitoring of the entire StudentHub portal:
• KPI Stat Metric Cards: Real-time calculation of Total Students, Active Campus Events, Total RSVP Registrations, and Security Audit Actions.
• Stored Procedure Execution: Calls MySQL stored procedure `CALL sp_registrations_by_month()` to compute month-by-month registration velocity, rendering pure CSS proportional bar charts.
• User Lifecycle Controls: Interactive table displaying registered campus users with 1-click status toggles ('Active' ↔ 'Inactive') and last login timestamps.
• Security Audit Trail (`admin/audit.php`): Full system audit trail logging actions (`login`, `register`, `student_create`, `event_delete`), actor email, IP address, and timestamps.

2. Implementation Code: Stored Procedure & Dynamic Analytics

Listing: admin/dashboard.php — Stored Procedure Execution and User Lifecycle Management

```
// admin/dashboard.php - Stored Procedure Invocation & Analytics
require_once __DIR__ . '/../includes/functions.php';
require_once __DIR__ . '/../includes/db.php';
require_role('admin'); // Restricts access exclusively to administrators

// Execute Stored Procedure
$monthly = $pdo->query('CALL sp_registrations_by_month()')->fetchAll();
$pdo->query('SELECT 1')->closeCursor(); // Free connection cursor for subsequent queries

// User Active / Inactive Status Toggle
if ($_SERVER['REQUEST_METHOD'] === 'POST' && ($_POST['do'] ?? '') === 'toggle') {
    if (csrf_check()) {
        $uid = (int)$_POST['user_id'];
        $u = $pdo->prepare('SELECT status, email FROM users WHERE id=?');
        $u->execute([$uid]);
        $row = $u->fetch();
        $next = $row['status'] === 'active' ? 'inactive' : 'active';
        $pdo->prepare('UPDATE users SET status=? WHERE id=?')->execute([$next, $uid]);
        audit($pdo, 'user_status_' . $next, 'User: ' . $row['email']);
        flash('success', 'User account ' . $next . ' successfully.');
    }
    redirect('dashboard.php');
}
```

3. Verification Screenshot: Executive Admin Dashboard

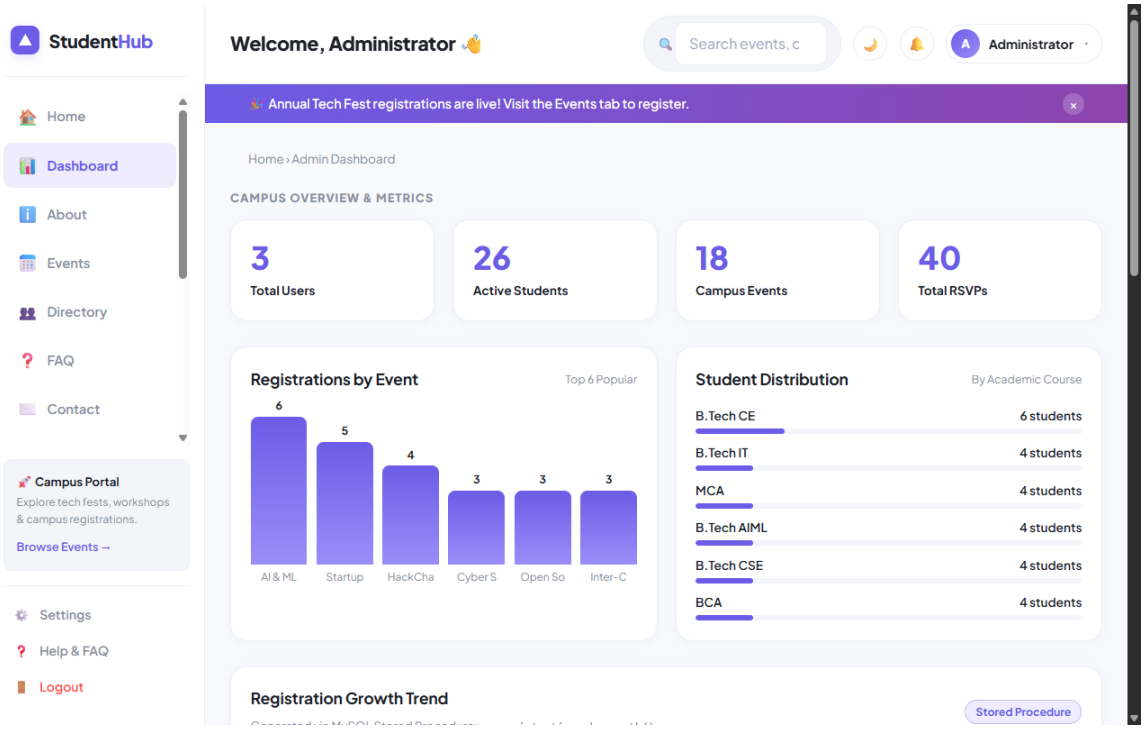


Figure: Admin Dashboard with KPI Stat Cards, Stored Procedure Monthly Chart, and User Status Toggles

4. Verification Screenshot: Security Audit Log Viewer

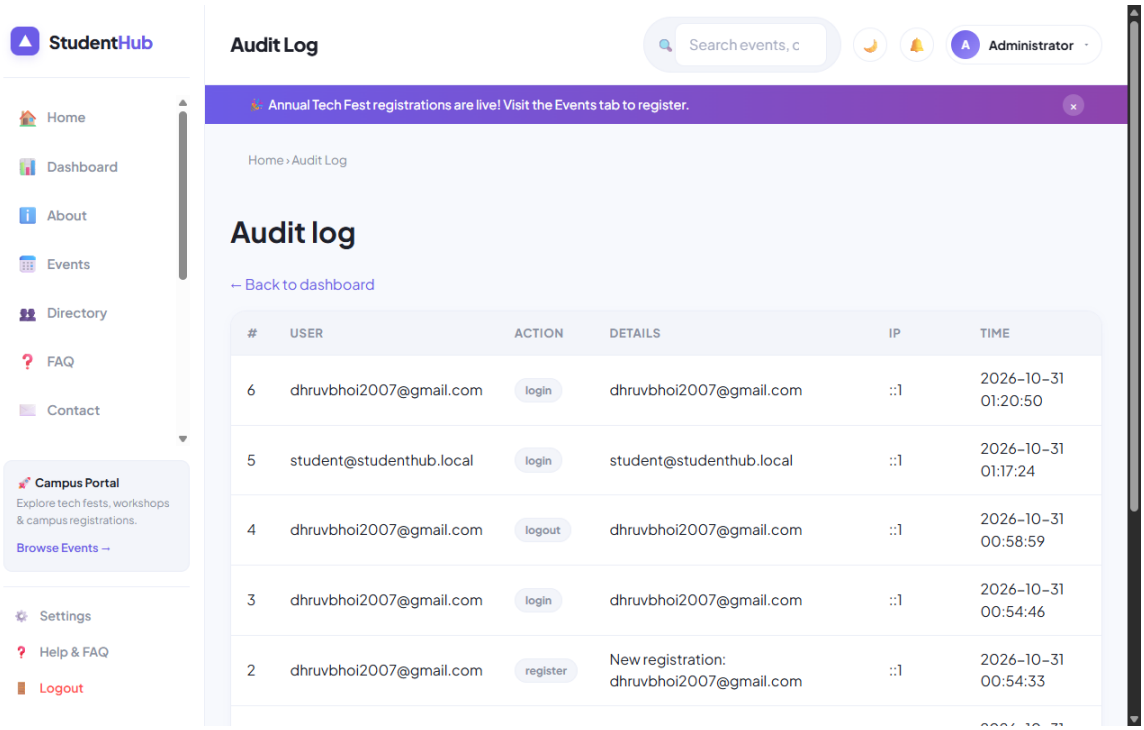


Figure: Immutable Security Audit Trail (admin/audit.php) Tracking User Actions and IP Addresses

5. Key Questions & Analysis

Q: How does `require_role('admin')` enforce access control across sensitive endpoints?

Ans: It checks `$_SESSION['user']['role'] === 'admin'`. If a student or guest attempts to access the script directly via URL, the function emits `HTTP 403 Forbidden`, dispatches an access denied flash message, and redirects the client to `dashboard.php`.

Q: Why are stored procedures used for aggregation queries like monthly registrations?

Ans: Stored procedures are pre-compiled and optimized on the MySQL database server. They minimize network latency by executing complex multi-table groupings internally, provide access abstraction, and enhance enterprise security.